

```

1  import torch.nn as nn
2  import torch
3
4  # Define DenseNet1D block (simplified)
5  class DenseBlock1D(nn.Module):
6      def __init__(self, in_channels, growth_rate, n_layers):
7          super().__init__()
8          self.layers = nn.ModuleList()
9          for i in range(n_layers):
10             self.layers.append(
11                 nn.Sequential(
12                     nn.BatchNorm1d(in_channels + i * growth_rate),
13                     nn.ReLU(inplace=True),
14                     nn.Conv1d(in_channels + i * growth_rate, growth_rate, 3, padding=1
15                             , bias=False)
16                 )
17             )
18         def forward(self, x):
19             features = [x]
20             for layer in self.layers:
21                 out = layer(torch.cat(features, dim=1))
22                 features.append(out)
23             return torch.cat(features, dim=1)
24
25  class DenseNet1D(nn.Module):
26      def __init__(self, csi_channels, meta_feature_dim, num_classes):
27          super().__init__()
28          self.initial_conv = nn.Conv1d(csi_channels, 64, 7, stride=2, padding=3)
29          self.dense_block = DenseBlock1D(64, growth_rate=32, n_layers=4)
30          self.transition = nn.Sequential(
31              nn.BatchNorm1d(64 + 4*32),
32              nn.ReLU(inplace=True),
33              nn.Conv1d(64 + 4*32, 128, 1),
34              nn.AvgPool1d(2)
35          )
36          self.global_pool = nn.AdaptiveAvgPool1d(1)
37
38          self.lstm = nn.LSTM(meta_feature_dim, 64, 2, batch_first=True, bidirectional=
39                             True)
40          self.fc = nn.Linear(128 + 64*2, num_classes)
41
42         def forward(self, csi_seq, meta_seq):
43             x = csi_seq.permute(0, 2, 1)
44             x = self.initial_conv(x)
45             x = self.dense_block(x)
46             x = self.transition(x)
47             x = self.global_pool(x).squeeze(-1)
48
49             _, (h_n, _) = self.lstm(meta_seq)
50             h_n = torch.cat([h_n[-2], h_n[-1]], dim=1)
51
52             combined = torch.cat([x, h_n], dim=1)
53             return self.fc(combined)
54

```